

SofaScore Local Lab

Projet Java / Spring Boot local sous Windows

Cadrage fonctionnel, architecture technique, gouvernance des accès, données, sécurité, tests et intégration future au projet global



Version	0.1.0	Date de référence	8 août 2026
Statut	Cadrage complémentaire - expérimentation locale contrôlée	Porteur	Projet individuel, un seul opérateur humain
Cible technique	Eclipse 2026-06, Spring Tools 5.3.0, Spring Boot 4.1.0, Java 25 LTS	Hébergement initial	Poste Windows local ; PostgreSQL dans Docker Desktop
Dépôt proposé	betting-sofascore-local-lab	Dépendance au VPS	Aucune

Document de travail destiné au cadrage du laboratoire local SofaScore comme complément non critique du Betting Project. Les endpoints, structures, conditions d'utilisation et comportements réseau peuvent évoluer sans préavis. Toute promotion hors du périmètre local exige une nouvelle décision d'architecture.

Fiche du document et avertissements

Champ	Contenu
Objet	Formaliser un laboratoire Java local capable d'interroger de manière contrôlée des endpoints SofaScore, de conserver les réponses, de normaliser certaines données et de préparer un export facultatif vers le Betting Project.
Public	Porteur du projet, futur intervenant technique et agents de développement travaillant à partir de Work Orders bornés.
Décision principale	Créer un dépôt isolé, exécutable uniquement sous Windows, utilisant Java 25 LTS et Spring Boot 4.1.0, sans dépendance critique du Betting Project.
Périmètre initial	Football, appels manuels, réponses brutes, normalisation limitée, comparaison de snapshots et export JSON.
Référentiel	Document de cadrage du Betting Project v0.5.1, complété par l'ADR-SS-001.
Horizon	Prototype local, benchmark technique, puis décision d'abandon, de maintien local ou d'intégration facultative.

Avertissement contractuel et juridique

Ce document ne constitue pas une validation juridique de l'automatisation d'accès aux données SofaScore. Les conditions de SofaScore peuvent restreindre l'usage de moyens automatisés, l'agrégation, l'extraction, l'intégration et la rétro-ingénierie. Elles doivent être relues avant toute évolution du périmètre. Le laboratoire doit s'arrêter en cas de refus, de challenge ou de blocage et ne doit intégrer aucun mécanisme de contournement.

Principe de confiance

Le laboratoire mesure, conserve et explique. Il ne transforme pas une accessibilité technique locale en droit d'usage, en SLA, en garantie de stabilité ou en autorisation de production. Les données issues de l'expérimentation restent facultatives et ne remplacent pas les fournisseurs principaux du Betting Project.

Évolution par rapport au cadrage v0.5.1

Le cadrage principal exclut les appels directs aux endpoints SofaScore depuis le cœur du Betting Project et le VPS, et privilégie un pont d'observation visuelle. Le présent document crée une exception limitée à un dépôt local distinct. Il ne modifie ni la non-dépendance du projet principal, ni l'absence de connecteur SofaScore sur le VPS, ni l'exigence de fonctionnement lorsque le poste Windows est éteint.

Synthèse exécutive

Le projet est techniquement cohérent avec l'environnement retenu. Spring Boot 4.1.0 exige Java 17 au minimum et déclare une compatibilité jusqu'à Java 26 incluse ; Java 25 est donc une cible supportée et présente l'avantage d'être une version LTS. Spring Tools 5.3.0 dispose d'une distribution pour Eclipse 2026-06 (4.40). **[R1–R5]**

Décision		Décision	
1	Dépôt séparé. Le laboratoire ne partage ni base, ni cycle de déploiement, ni responsabilité opérationnelle avec le coeur du Betting Project.	2	Windows local uniquement. L'application Java tourne directement sur le poste ; PostgreSQL peut rester dans Docker Desktop.
3	Java 25 LTS. La compilation et l'exécution utilisent la même version LTS, plutôt qu'un JDK 26 avec cible inférieure.	4	RestClient synchrone. Un trafic volontairement faible ne justifie ni WebFlux, ni une architecture réactive.
5	Interface web locale. Spring MVC et Thymeleaf fournissent une GUI simple, réutilisable et packagée dans le même JAR.	6	Réseau désactivé par défaut. Appels manuels, concurrence maximale de 1, cache, délai minimal, arrêt sur 403/429/challenge.
7	Brut et normalisé. Chaque réponse conserve son statut, sa latence, son hash et sa version de parseur avant toute normalisation.	8	Intégration par contrat. Le seul lien futur autorisé est un export JSON versionné, puis éventuellement un push HTTPS sortant.

Statut cible

EXPERIMENTALLOCAL ONLYNOT PRODUCTION APPROVEDNO CRITICAL DEPENDENCY

Le MVP doit démontrer qu'une requête manuelle peut être tracée, rejouée hors ligne, normalisée et exportée sans compromettre l'isolation. Le polling live, la collecte massive et l'intégration directe au VPS sont volontairement repoussés après une décision formelle fondée sur des mesures.

Table des matières

Fiche du document et avertissements	2
Synthèse exécutive	3
1 Contexte, objectif et relation avec le Betting Project	6
1.1 Besoin exprimé	6
1.2 Position du cadrage principal	6
1.3 Évolution de gouvernance	6
1.4 Nom, dépôt et identité du projet	6
2 Périmètre fonctionnel et non-objectifs	7
2.1 Périmètre du MVP	7
2.2 Écrans proposés	7
2.3 Non-objectifs des premières versions	7
3 Environnement de travail et compatibilité	8
3.1 Versions retenues	8
3.2 Décision Java 25 LTS	8
3.3 Configuration Eclipse recommandée	8
3.4 Répartition Windows / Docker / WSL2	9
4 Architecture cible	9
4.1 Vue d'ensemble	9
4.2 Principes architecturaux	9
4.3 Topologie de déploiement initiale	10
4.4 Flux d'une requête	10
5 Choix techniques	10
5.1 Client HTTP : RestClient	10
5.2 Threads virtuels	10
5.3 Interface graphique : Spring MVC + Thymeleaf	11
5.4 Base, migrations et cache	11
5.5 Packaging	11
6 Organisation du code et contrats internes	11
6.1 Structure proposée	11
6.2 Ports principaux	12
6.3 DTO et parseurs versionnés	12
6.4 Séparation des responsabilités	13
7 Dépendances Maven et configuration	13
7.1 Dépendances recommandées	13
7.2 Extrait Maven indicatif	13
7.3 Configuration locale de référence	14
8 Catalogue d'endpoints et politique réseau	14
8.1 Catalogue logique	14
8.2 Garde-fous avant appel	15
8.3 Comportement selon la réponse	15
8.4 Interdictions techniques	16
9 Modèle de données, provenance et rétention	16
9.1 Snapshot fournisseur	16
9.2 Données normalisées	16
9.3 Provenance	16
9.4 Rétention	17
10 Interface utilisateur locale	17
10.1 Principes de conception	17
10.2 Tableau de bord	17
10.3 Expérience d'un appel manuel	18
10.4 Widget officiel	18
11 Intégration future avec le Betting Project	18
11.1 Phase 1 : export fichier	18
11.2 Phase 2 éventuelle : push sortant	18
11.3 Rôle analytique futur	19

12 Sécurité, conditions d'utilisation et maîtrise du risque	19
12.1 Position de prudence	19
12.2 Mesures de sécurité	19
12.3 Matrice de risques	19
12.4 Journalisation	20
13 Stratégie de tests	20
13.1 Principe hors ligne d'abord	20
13.2 Matrice de tests	20
13.3 Niveaux et outils	21
13.4 Séparation des tests réels	21
13.5 Critères de qualité des fixtures	21
14 Exploitation locale, supervision et sauvegarde	21
14.1 Health et métriques	21
14.2 Procédures minimales	22
14.3 Mode dégradé	22
15 Feuille de route	22
16 Critères d'acceptation	23
16.1 Critères de décision après benchmark	24
17 Décisions finales et livrables suivants	24
17.1 Décisions à figer	24
17.2 Livrables suivants	24
18 Annexe A - Exemple de modèle de données	26
19 Annexe B - Exemples de configurations et commandes	27
19.1 Variables d'environnement locales	27
19.2 Commandes de contrôle	27
19.3 Arborescence documentaire	27
20 Annexe C - Références	28

1. Contexte, objectif et relation avec le Betting Project

1.1 Besoin exprimé

Le laboratoire doit permettre d'étudier plus profondément la richesse de SofaScore depuis l'environnement local Windows, où les blocages associés aux adresses de datacenters peuvent être différents de ceux rencontrés sur le VPS. L'objectif n'est pas de remplacer immédiatement les fournisseurs déjà retenus, mais de produire des faits mesurés sur :

- l'accessibilité et la latence depuis la connexion locale ;
- les familles de données réellement disponibles ;
- la stabilité des identifiants et des schémas JSON ;
- la fréquence de mise à jour et les corrections tardives ;
- la capacité à conserver, rejouer et comparer les réponses ;
- la valeur d'un enrichissement facultatif pour les analyses du Betting Project.

1.2 Position du cadrage principal

Le document de cadrage v0.5.1 définit deux règles structurantes :

1. le système de production doit fonctionner sans le poste Windows ;
2. le code du Betting Project et le VPS ne doivent pas dépendre d'appels directs à des endpoints SofaScore non documentés.

Il propose à la place un *SofaScore Observation Bridge* : consultation normale dans le navigateur, capture ou saisie locale, extraction assistée, validation humaine, puis transmission d'une observation JSON non critique. Il distingue explicitement les `VisualObservation` des `ProviderSnapshot`. **[R0]**

1.3 Évolution de gouvernance

Exception limitée, pas renversement de la règle globale

L'ADR-SS-001 autorise un accès direct uniquement dans un laboratoire local séparé. Le coeur du Betting Project reste conforme au cadrage v0.5.1 : aucun appel direct depuis le VPS, aucune dépendance critique, aucune exigence que le poste Windows soit disponible et aucune promotion implicite des données expérimentales.

Sujet	Betting Project principal	SofaScore Local Lab
Exécution	VPS permanent, indépendant du poste Windows	Windows local uniquement
Mode SofaScore	Observation visuelle facultative	Endpoints configurés dans un adaptateur expérimental
Dépendance	Aucune dépendance critique	Source étudiée, désactivable et non critique
Stockage	PostgreSQL source de vérité du projet global	PostgreSQL local séparé
Production	Oui, après jalons et validation	Non approuvée
Intégration	Modèle canonique multi-fournisseurs	Export JSON normalisé uniquement

1.4 Nom, dépôt et identité du projet

Nom fonctionnel : SofaScore Local Lab
Dépôt Git : betting-sofascore-local-lab
Application : sofascore-local-lab
Package racine proposé : com.geoffrey.betting.sofascorelocal

Le dépôt doit contenir ses propres ADR, Work Orders, fixtures, migrations, procédures de démarrage et rapports de benchmark. Il ne doit pas être un sous-module indispensable au build du Betting Project.

2. Périmètre fonctionnel et non-objectifs

2.1 Périmètre du MVP

Le premier périmètre est volontairement étroit : football, appels déclenchés par l'opérateur, navigation par date et événement, conservation des réponses, normalisation limitée et export contrôlé.

1. Rechercher les événements d'une date ou d'un identifiant connu.
2. Afficher les informations principales : compétition, participants, horaire, statut et score.
3. Charger manuellement les détails, statistiques, incidents ou compositions lorsque disponibles.
4. Enregistrer la réponse brute, le statut HTTP, le type de contenu, la latence et l'empreinte SHA-256.
5. Parser les champs explicitement pris en charge et signaler les champs inconnus ou absents.
6. Comparer deux instantanés d'un même événement.
7. Exporter une enveloppe JSON versionnée avec provenance et avertissements.

2.2 Écrans proposés

Écran	Fonction principale	Informations clés
Tableau de bord	État du laboratoire	Connectivité, activation réseau, dernier appel, dernier statut HTTP, circuit ouvert/fermé, nombre de snapshots et erreurs récentes.
Recherche	Localiser les événements	Date, sport, compétition, identifiant d'événement, paramètres autorisés.
Événements	Parcourir les résultats	Participants, compétition, heure locale/UTC, statut, score et disponibilité des familles de données.
Détail	Consulter un événement	Données d'identité, saison, phase, score, contexte et liens vers les snapshots.
Statistiques	Visualiser les métriques	Statistiques d'équipe, incidents, compositions et joueurs selon disponibilité.
Réponses brutes	Auditer le transport	JSON reçu, horodatage, hash, headers autorisés, latence et version de parseur.
Historique	Comparer les snapshots	Champs ajoutés, supprimés, modifiés, corrections de score et différences temporelles.
Export	Préparer l'intégration	Aperçu JSON canonique, validation, avertissements, version de schéma et écriture dans le dossier local.
Paramètres	Piloter les garde-fous	Activation, délai minimal, rétention, endpoints logiques autorisés, emplacement des exports.

2.3 Non-objectifs des premières versions

- polling live permanent ou surveillance de toutes les rencontres ;
- collecte historique massive ;
- recherche de cotes ou calcul de valuebet ;
- recommandation de pari ou prise de pari automatisée ;
- scraping HTML, OCR, extension de navigateur ou simulation de navigateur ;
- rotation de proxy, changement automatique d'adresse ou contournement d'un challenge ;
- exécution sur le VPS ou ouverture d'un port entrant vers Windows ;
- frontend React/Angular séparé ;
- application mobile native ;

— dépendance du rapport quotidien du Betting Project aux données SofaScore.

Critère de sortie du MVP

Le MVP n'est pas validé parce qu'une page affiche des données. Il est validé lorsque les appels sont gouvernés, les réponses sont traçables et rejouables, les erreurs sont maîtrisées, les schémas incompatibles sont détectés et l'arrêt du laboratoire n'a aucun effet sur le projet principal.

3. Environnement de travail et compatibilité

3.1 Versions retenues

Composant	Version / choix	Décision de cadrage
Système local	Windows 11	Exécution directe de l'application Java.
Eclipse	2026-06 (4.40.0), build 20260604-0653	IDE principal. La branche 4.40 correspond à la livraison Eclipse 2026-06. [R4–R5]
Spring Tools	5.3.0.202607290337	Extension Spring retenue ; la distribution 5.3.0 cible Eclipse 4.40. [R4]
Java	25 LTS	Compilation et runtime. Java 26 est abandonné comme cible. [R1–R2]
Spring Boot	4.1.0	Compatible avec Java 17 à 26 inclus ; Java 25 est donc dans la plage supportée. [R3]
Spring Framework	Version gérée par Boot 4.1	Ne pas forcer manuellement la version ; Boot 4.1 exige Spring Framework 7.0.8 ou supérieur. [R3]
Build	Maven, wrappers versionnés	Les commandes doivent fonctionner avec <code>mvnw.cmd</code> et <code>mvnw</code> .
Base locale	PostgreSQL dans Docker Desktop	Alignement avec le modèle cible du Betting Project.

3.2 Décision Java 25 LTS

Oracle classe Java 25 parmi les versions LTS, alors que Java 26 appartient au cycle non-LTS et est rapidement supplanté par la version suivante. La cible Java 25 réduit le risque de migration semestrielle et donne un socle cohérent pour le build, l'exécution locale, les images de test et les futures intégrations. [\[R1–R2\]](#)

Cible de compilation et d'exécution

Le projet utilise un JDK 25 comme JRE système dans Eclipse, comme JDK Maven et comme runtime de l'application. La propriété Maven recommandée est `<java.version>25</java.version>`. Il n'est pas nécessaire de conserver Java 26 pour ce laboratoire.

3.3 Configuration Eclipse recommandée

- enregistrer un JDK 25 dans *Installed JREs* et le définir comme défaut du workspace ;
- régler le niveau de compilation du projet sur 25 ;
- importer le dépôt comme projet Maven existant ;
- utiliser le wrapper Maven dans les procédures et dans l'intégration continue ;
- créer un profil de lancement local avec le profil `Spring local` ;
- conserver les secrets et chemins locaux dans des variables d'environnement ou un fichier non versionné.

3.4 Répartition Windows / Docker / WSL2

Composant	Rôle
Windows	Eclipse, JVM, application Spring Boot, navigateur, Git, outils de diagnostic et dossier d'exports.
Docker Desktop	PostgreSQL local et, si nécessaire, outils de test éphémères.
WSL2	Scripts Bash, validation Linux facultative, vérification des exports et compatibilité future ; aucune exécution normale obligatoire.
VPS	Hors périmètre du laboratoire ; ne reçoit pas les appels SofaScore.

4. Architecture cible

4.1 Vue d'ensemble

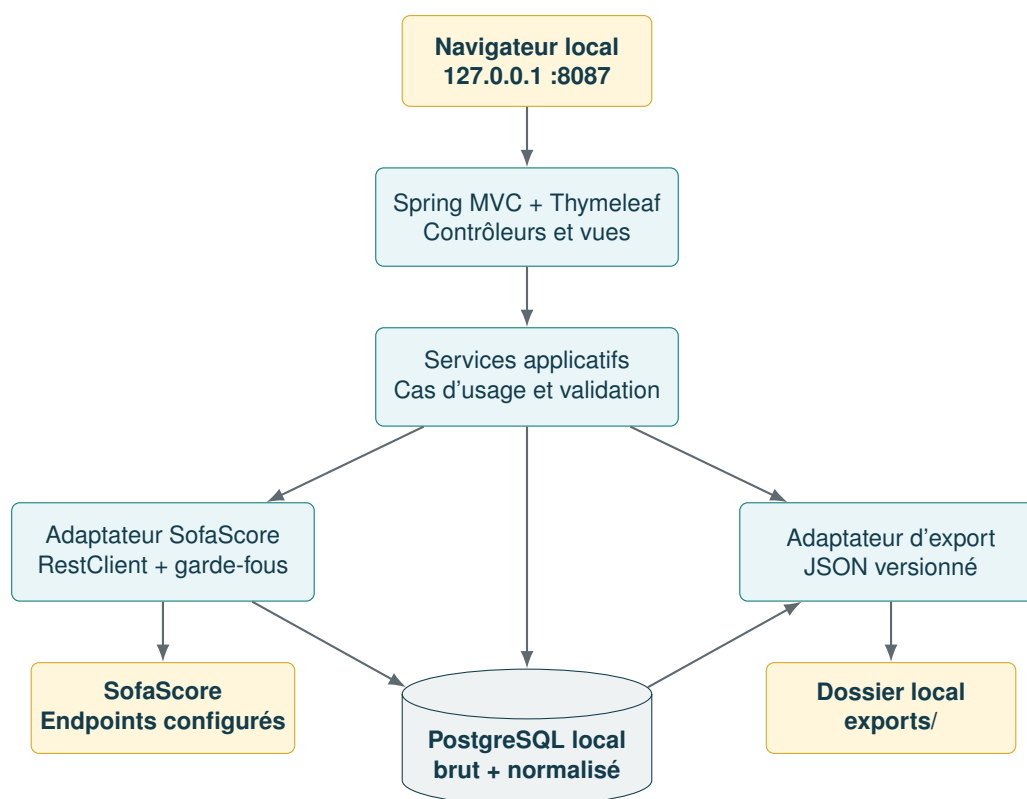


FIGURE 1 – Architecture locale : interface, services, adaptateur fournisseur, stockage séparé et export contrôlé.

4.2 Principes architecturaux

1. **Isolation du fournisseur.** Le domaine n'appelle jamais une URI SofaScore ; l'adaptateur traduit les types logiques et les DTO externes.
2. **Double stockage.** Le payload brut est conservé avant normalisation afin de permettre audit, replay et correction du parseur.
3. **Désactivation simple.** Une propriété unique peut neutraliser tous les appels sortants sans empêcher l'accès aux snapshots locaux.
4. **Trafic faible.** Une seule requête simultanée, appels manuels et cache obligatoire.
5. **Interface locale seulement.** Le serveur écoute sur 127.0.0.1 et n'est pas publié sur le réseau local ou Internet.
6. **Export à sens unique.** Les données quittent le laboratoire uniquement sous forme normalisée et versionnée.

4.3 Topologie de déploiement initiale

```
Windows 11
|-- Eclipse 2026-06 + Spring Tools 5.3
|-- JDK 25 LTS
|-- sofascore-local-lab.jar
|-- navigateur -> http://127.0.0.1:8087
|-- exports/
'-- Docker Desktop
    '-- PostgreSQL
```

L'application Java n'est pas conteneurisée pendant le benchmark initial. Ce choix simplifie le débogage Eclipse, l'observation des erreurs réseau et l'arrêt immédiat. Un packaging Docker pourra être étudié seulement si la reproductibilité l'exige, sans modifier la règle *local only*.

4.4 Flux d'une requête

1. l'opérateur sélectionne un endpoint logique et des paramètres autorisés ;
2. le service vérifie l'activation, le cache, le délai minimal et l'état du circuit ;
3. l'adaptateur construit la requête et mesure la latence ;
4. le statut, les métadonnées et le payload sont enregistrés ;
5. le hash détermine si le snapshot est nouveau ;
6. un parseur versionné transforme les champs connus ;
7. les contrôles de cohérence attribuent un statut de validation ;
8. l'interface affiche le résultat et les avertissements ;
9. un export peut être produit explicitement.

5. Choix techniques

5.1 Client HTTP : RestClient

Spring Framework présente `RestClient` comme le client synchrone moderne. Depuis Spring Framework 7, `RestTemplate` est déprécié au profit de `RestClient`. [R6]

Pourquoi un client synchrone

Le laboratoire limite volontairement la concurrence à un seul appel. `RestClient` offre une API fluide, une intégration naturelle à Spring Boot, une instrumentation simple et un modèle d'erreur lisible. `WebClient` et `Reactor` n'apporteraient pas de bénéfice proportionné dans le MVP.

L'implémentation doit définir explicitement :

- les timeouts de connexion et de lecture ;
- les handlers de statut ;
- la taille maximale de réponse acceptée ;
- le type de contenu attendu ;
- les métriques de latence et de résultat ;
- les logs expurgés.

5.2 Threads virtuels

Spring Boot 4.1 peut auto-configurer des exécuteurs et planificateurs utilisant des threads virtuels avec Java 21+ lorsque `spring.threads.virtual.enabled=true`. [R7]

Le laboratoire peut activer cette propriété pour simplifier l'exécution bloquante, mais elle ne doit pas être confondue avec une permission d'augmenter la concurrence réseau. Le sémaphore ou garde-fou applicatif reste fixé à 1.

5.3 Interface graphique : Spring MVC + Thymeleaf

La GUI recommandée est une interface web rendue côté serveur :

- aucune chaîne frontend séparée ;
- même projet, même JAR et même modèle de sécurité ;
- tableaux, formulaires, diff JSON et historique faciles à présenter ;
- débogage direct depuis Eclipse ;
- réutilisation future possible dans une interface d'administration plus large.

JavaFX reste une alternative ultérieure pour une application de bureau native, mais il ajouterait une couche de packaging, des dépendances graphiques et une seconde architecture de présentation. Les widgets officiels SofaScore peuvent éventuellement être affichés dans un onglet de contrôle visuel, mais ils ne deviennent pas une API métier. [\[R10\]](#)

5.4 Base, migrations et cache

Technologie	Rôle et justification
PostgreSQL	JSONB, contraintes, historique, requêtes de comparaison et alignement avec le projet global.
Spring Data JPA	CRUD et relations pour les entités normalisées ; les payloads volumineux peuvent utiliser JDBC/JSONB ciblé si nécessaire.
Flyway	Migrations versionnées et reproductibles, y compris dans Testcontainers.
Caffeine	Cache local en mémoire pour référentiels et réponses récentes ; Redis n'est pas justifié.
Actuator	Health, info et métriques locales ; exposition limitée à l'interface de boucle locale.

5.5 Packaging

Le livrable initial est un JAR exécutable. Les procédures doivent couvrir :

```
.\mvnw.cmd clean verify
.\mvnw.cmd spring-boot:run -Dspring-boot.run.profiles=local
java -jar target\sofascore-local-lab-0.1.0.jar --spring.profiles.active=local
```

6. Organisation du code et contrats internes

6.1 Structure proposée

```
src/main/java/com/geoffrey/betting/sofascorelocal/
|-- SofascoreLocalApplication.java
|-- config/
|   |-- SofascoreProperties.java
|   |-- HttpClientConfiguration.java
|   '-- SchedulingConfiguration.java
|-- domain/
|   |-- event/
|   |-- participant/
|   |-- competition/
|   |-- snapshot/
|   '-- export/
|-- application/
|   |-- SearchEventsUseCase.java
|   |-- LoadEventDetailsUseCase.java
|   |-- LoadEventStatisticsUseCase.java
|   |-- CompareSnapshotsUseCase.java
|   '-- ExportEventDataUseCase.java
|-- port/
|   |-- SofascoreDataProvider.java
```

```

| |-- SnapshotRepository.java
| |-- ExportPublisher.java
|-- adapter/
| |-- sofascore/
| | |-- SofascoreRestClientAdapter.java
| | |-- SofascoreEndpointCatalog.java
| | |-- dto/
| | |-- mapper/
| |-- persistence/
| |-- export/
| |-- web/
|-- support/
| |-- hashing/
| |-- clock/
|-- json/

```

6.2 Ports principaux

```

public interface SofascoreDataProvider {
    ProviderResponse load(EndpointRequest request);
}

public interface SnapshotRepository {
    Optional<ProviderSnapshot> findByPayloadHash(String sha256);
    ProviderSnapshot save(ProviderSnapshot snapshot);
}

public interface ExportPublisher {
    Path publish(CanonicalExport export);
}

```

Le domaine connaît des concepts tels que Event, Competition, Metric, Snapshot et Export. Il ne connaît ni les chemins HTTP, ni les noms exacts de champs SofaScore, ni les headers de transport.

6.3 DTO et parseurs versionnés

Chaque famille de réponse possède :

- un DTO externe minimal, tolérant les champs supplémentaires ;
- un parseur versionné ;
- un mapper vers le modèle interne ;
- des fixtures anonymisées ou publiques ;
- des tests de champs manquants, de types modifiés et de valeurs hors bornes.

Conservation avant interprétation

Une réponse reçue avec succès est stockée avant de tenter la normalisation. Une erreur de parseur ne doit pas faire perdre la preuve brute ni conduire à rappeler immédiatement la source.

6.4 Séparation des responsabilités

Couche	Responsabilité
Web	Validation des formulaires, présentation, pagination et commandes explicites.
Application	Orchestration des cas d'usage, idempotence, règles d'activation et transactions.
Domaine	Modèle canonique local, états, cohérence et politiques indépendantes du fournisseur.
Adaptateur SofaScore	URI, DTO, mapping, transport, statuts HTTP et restrictions spécifiques.
Persistence	Entités, requêtes, migrations et politique de rétention.
Export	Schéma de sortie, version, validation et écriture atomique.
Support	Hash, horloge injectable, JSON canonique et utilitaires techniques.

7. Dépendances Maven et configuration

7.1 Dépendances recommandées

Spring Boot 4.1 fournit plusieurs starters spécialisés. Le socle retient les starters dédiés au client REST, à Spring MVC et aux migrations Flyway. Le starter historique web est déprécié au profit du starter MVC spécialisé. [\[R8\]](#)

Dépendance	Usage
spring-boot-starter-webmvc	Interface web locale et contrôleurs MVC.
spring-boot-starter-restclient	Client HTTP bloquant et auto-configuration associée.
spring-boot-starter-thymeleaf	Templates HTML rendus côté serveur.
spring-boot-starter-validation	Validation des formulaires et propriétés.
spring-boot-starter-data-jpa	Persistence des données normalisées.
spring-boot-starter-flyway	Migrations de base.
flyway-database-postgresql	Support PostgreSQL de Flyway.
postgresql	Driver JDBC.
spring-boot-starter-cache	Abstraction de cache.
caffeine	Implémentation locale du cache.
spring-boot-starter-actuator	Health, info et métriques.
spring-boot-starter-test	Tests Spring Boot et JUnit gérés par le BOM.
spring-boot-starter-restclient-test	Tests du client HTTP bloquant.
testcontainers-postgresql	Base PostgreSQL éphémère pour les tests d'intégration.

7.2 Extrait Maven indicatif

```
<properties>
  <java.version>25</java.version>
</properties>

<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-restclient</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
```

```

    <artifactId>spring-boot-starter-thymeleaf</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-flyway</artifactId>
  </dependency>
</dependencies>

```

7.3 Configuration locale de référence

```

server:
  address: 127.0.0.1
  port: 8087

spring:
  threads:
    virtual:
      enabled: true
  jpa:
    open-in-view: false
  datasource:
    url: ${SOFAScore_DB_URL}
    username: ${SOFAScore_DB_USER}
    password: ${SOFAScore_DB_PASSWORD}

sofascore:
  enabled: false
  base-url: ${SOFAScore_BASE_URL}
  maximum-concurrency: 1
  minimum-delay: 3s
  automatic-refresh-enabled: false
  live-polling-enabled: false
  store-raw-payloads: true
  raw-payload-retention-days: 30
  export-directory: ${SOFAScore_EXPORT_DIR:./exports}

```

Valeurs prudentes, pas quotas officiels

Le délai de 3 secondes, la concurrence de 1 et la rétention de 30 jours sont des règles internes de démarrage. Elles ne décrivent ni un droit d'accès, ni une capacité publiée, ni une promesse de stabilité. Toute évolution doit être fondée sur les résultats du benchmark et un nouvel arbitrage.

8. Catalogue d'endpoints et politique réseau

8.1 Catalogue logique

Les chemins réels sont confinés dans `SofascoreEndpointCatalog`. Le reste du code utilise des types logiques :

```

public enum SofascoreEndpointType {
    SCHEDULED_EVENTS,
    EVENT_DETAILS,
    EVENT_STATISTICS,
    EVENT_INCIDENTS,
    EVENT_LINEUPS,
    TOURNAMENT_STANDINGS,
    TEAM_RECENT_EVENTS
}

```

Type logique	Cache initial	Déclenchement	Usage
SCHEDULED_EVENTS	10 min	Manuel	Recherche par date et vérification des identités.
EVENT_DETAILS	15 min	Manuel	Détail d'une rencontre retenue.
EVENT_STATISTICS	30 min après match	Manuel	Statistiques d'équipe selon disponibilité.
EVENT_INCIDENTS	15 min	Manuel	Buts, cartons, remplacements et autres incidents.
EVENT_LINEUPS	15 min	Manuel	Titulaires, bancs et formations.
TOURNAMENT_ STANDINGS	6 h	Manuel	Contexte de classement.
TEAM_RECENT_EVENTS	1 h	Manuel	Forme récente, sans collecte historique massive.

Chaque entrée du catalogue doit exposer : URI modèle, paramètres autorisés, parseur, cache, délai minimal, niveau de sensibilité, taille maximale, activation et date de dernière validation.

8.2 Garde-fous avant appel

Avant chaque requête, l'application vérifie dans cet ordre :

1. le connecteur global est activé ;
2. l'endpoint logique est autorisé ;
3. les paramètres passent les validations ;
4. aucune réponse fraîche n'est déjà disponible dans le cache ;
5. le délai minimal depuis le dernier appel est respecté ;
6. aucun circuit n'est ouvert ;
7. le sémaphore de concurrence peut être acquis ;
8. l'opérateur a confirmé l'appel réel si le profil le requiert.

8.3 Comportement selon la réponse

Résultat	Comportement attendu
200 JSON	Enregistrer les métadonnées et le payload, calculer le hash, dédupliquer, parser puis afficher les avertissements.
304	Réutiliser le snapshot associé si le mécanisme de cache conditionnel est explicitement pris en charge.
400	Erreur de requête ; aucun retry automatique.
401/403	Suspendre le connecteur, ouvrir le circuit et demander une décision humaine.
429	Lire Retry-After s'il existe, suspendre les appels, journaliser et interdire tout retry rapide.
5xx	Au maximum un ou deux retries espacés, uniquement si la requête est idempotente et si le circuit reste fermé.
Timeout	Conserver l'erreur et la latence ; retry borné, jamais en boucle.
HTML inattendu	Classer la réponse comme challenge ou contenu non conforme, arrêter le traitement et suspendre le connecteur.
JSON inconnu	Conserver le brut, marquer SCHEMA_INCOMPATIBLE et ne pas normaliser silencieusement.
Payload trop grand	Interrompre ou rejeter selon la limite configurée, sans écrire une réponse partielle comme valide.

8.4 Interdictions techniques

Le code ne doit contenir ni rotation de proxy, ni changement automatique de User-Agent pour éviter un blocage, ni simulation de navigateur, ni réutilisation de cookies ou jetons interceptés, ni automatisation permanente du navigateur, ni stratégie visant à changer d'adresse après un blocage. Un refus explicite met fin aux appels jusqu'à décision humaine.

9. Modèle de données, provenance et rétention

9.1 Snapshot fournisseur

La table principale conserve la preuve brute indépendamment du succès du parseur :

```
provider_snapshot
-----
id
provider
logical_endpoint
request_key
requested_at
received_at
http_status
content_type
latency_ms
payload_jsonb
payload_sha256
parser_version
schema_status
error_code
created_at
```

Une contrainte unique ou une règle d'idempotence doit éviter de stocker plusieurs fois le même contenu pour la même clé de requête lorsque cela n'apporte aucune information temporelle utile.

9.2 Données normalisées

Le modèle initial peut comprendre :

— competition	— event_metric
— season	— event_incident
— participant	— event_lineup
— event	— provider_entity_mapping
— event_participant	— export_manifest

Les données normalisées doivent toujours référencer le snapshot d'origine, le parseur et l'heure de réception. Une correction ultérieure crée une nouvelle version plutôt qu'une modification invisible.

9.3 Provenance

```
source = SOFASCORE
acquisitionMode = DIRECT_LOCAL_ENDPOINT
receivedAt
sourceUpdatedAt
payloadHash
parserVersion
validationStatus
```

Statuts proposés :

RAW ONLY PARSED COHERENCE CHECKED HUMAN VALIDATED REJECTED

Séparation avec les observations visuelles

Une donnée acquise par endpoint local n'est pas une `VisualObservation`. Elle porte le mode `DIRECT_LOCAL_ENDPOINT`. Cette distinction doit rester visible dans la base, les exports et les futures analyses afin d'éviter de confondre les preuves, les règles de validation et les risques associés.

9.4 Rétention

Objet	Rétention initiale	Règle
Payloads bruts	30 jours	Configurable ; purge seulement après vérification des exports et fixtures nécessaires.
Métadonnées d'appel	12 mois	Conserver statut, latence, hash, endpoint logique et erreur, même après purge du payload si nécessaire.
Données normalisées	Jusqu'à décision	Versionnées et reliées à leur provenance.
Fixtures de test	Versionnées dans Git si autorisées et minimisées	Ne pas y inclure de secrets, cookies, identifiants de session ou données inutiles.
Exports	Conservation locale organisée	Nom, hash, version de schéma et manifeste.
Logs applicatifs	Rotation courte	Masquage des secrets et absence de payload complet dans les logs.

10. Interface utilisateur locale

10.1 Principes de conception

L'interface doit être un outil de contrôle, pas un déclencheur opaque. Toute action réseau réelle affiche clairement : l'endpoint logique, les paramètres, la présence d'un cache, la dernière requête et le risque d'appel externe.

- bouton d'activation générale avec état persistant ;
- bannière visible lorsque le réseau est actif ;
- confirmation avant appel réel dans les profils sensibles ;
- bouton d'arrêt global ;
- signalement immédiat de 403, 429, HTML inattendu ou schéma incompatible ;
- accès aux payloads bruts sans les injecter dans les logs ;
- comparaison structurée de snapshots ;
- aperçu et validation de l'export.

10.2 Tableau de bord

Le tableau de bord présente au minimum :

Bloc	Contenu
Connecteur	Activé/désactivé, base URL renseignée, endpoint autorisé, circuit, concurrence et délai restant.
Dernier appel	Heure, endpoint logique, statut HTTP, latence, taille et hash.
Santé	Base, espace disque, migrations, export, cache et Actuator.
Incidents	403, 429, timeouts, réponses HTML, incompatibilités de schéma et erreurs de persistance.
Données	Nombre de snapshots, événements, exports, payloads à purger et fixtures disponibles.

10.3 Expérience d'un appel manuel

1. l'opérateur recherche un événement ;
2. l'interface affiche les données locales disponibles ;
3. l'opérateur sélectionne une famille de données ;
4. l'application indique si le cache évite l'appel ;
5. si un appel reste nécessaire, un résumé est présenté ;
6. l'opérateur confirme ;
7. la réponse est stockée et l'écran affiche le statut, la latence, le hash, le parseur et les avertissements ;
8. l'opérateur peut comparer, valider ou exporter.

10.4 Widget officiel

Les widgets officiels sont des iframes destinées à l'affichage de statistiques mises à jour. Ils peuvent servir de référence visuelle dans une page distincte, mais la page parente ne doit pas les traiter comme une source métier ni tenter d'en extraire les données. [R10]

11. Intégration future avec le Betting Project

11.1 Phase 1 : export fichier

Le premier contrat d'intégration est un fichier JSON écrit atomiquement dans un dossier local :

```
{
  "schemaVersion": "1.0",
  "source": "SOFAScore",
  "acquisitionMode": "DIRECT_LOCAL_ENDPOINT",
  "sourceEventId": "123456",
  "observedAt": "2026-08-08T15:30:00Z",
  "receivedAt": "2026-08-08T15:30:01Z",
  "payloadHash": "sha256:...",
  "parserVersion": "1",
  "validationStatus": "HUMAN_VALIDATED",
  "event": {},
  "metrics": [],
  "warnings": []
}
```

Le fichier doit être accompagné d'un manifeste contenant nom, taille, hash, date, schéma et statut de validation. Le Betting Project n'est pas obligé de l'importer.

11.2 Phase 2 éventuelle : push sortant



Cette phase exige un ADR supplémentaire. Les règles minimales sont :

- aucune ouverture de port entrant sur Windows ;
- authentification dédiée et secret à privilège minimal ;
- aucun payload brut, cookie ou paramètre technique SofaScore transmis ;
- import idempotent par hash et version de schéma ;
- quarantaine des données incompatibles ;
- possibilité de désactiver le flux sans dégrader le projet global.

11.3 Rôle analytique futur

Après validation, les données SofaScore pourraient :

- compléter une analyse prématch prioritaire ;
- confirmer une divergence entre fournisseurs ;
- fournir des métriques non disponibles ailleurs ;
- enrichir un bilan post-match ;
- alimenter des jeux de tests ou des travaux de modélisation, sous réserve de provenance et de conditions d'usage.

Elles ne doivent pas devenir l'unique source de découverte, d'identité, de score final, de règlement ou de déclenchement live autonome.

12. Sécurité, conditions d'utilisation et maîtrise du risque

12.1 Position de prudence

Le site et l'application SofaScore fournissent des scores, statistiques et contenus sportifs. Les conditions officielles signalent des restrictions concernant notamment l'usage automatisé, l'extraction, l'agrégation, l'intégration et la rétro-ingénierie ; elles réservent également la possibilité de limiter ou bloquer l'accès. [R9]

Interprétation du laboratoire

Le laboratoire documente un risque accepté pour un prototype personnel et local. Il ne qualifie pas les endpoints étudiés d'API publique documentée, ne garantit pas leur licéité dans tous les scénarios et ne remplace pas une analyse contractuelle ou juridique.

12.2 Mesures de sécurité

Risque	Mesure de maîtrise
Exposition réseau	Écoute sur 127.0.0.1, pare-feu local, aucun reverse proxy public.
Secrets	Variables d'environnement ou coffre local ; jamais dans Git, les captures ou les logs.
Payload sensible	Conservation locale, accès limité, rétention configurable et absence d'envoi automatique.
Blocage / anti-bot	Arrêt sur 403, 429 ou challenge ; aucun contournement.
Boucle de retry	Nombre maximal explicite, backoff et circuit breaker.
Collecte excessive	Appels manuels, concurrence de 1, cache et liste fermée d'endpoints.
Schéma modifié	Statut d'incompatibilité, aucune normalisation silencieuse et replay hors ligne.
Dépendance opérationnelle	Projet principal continue sans Windows et sans SofaScore.
Export compromis	Hash, schéma, manifeste, validation et écriture atomique.

12.3 Matrice de risques

Risque	Probabilité	Impact	Réponse
Blocage de l'adresse locale	Moyenne	Moyen	Arrêt immédiat, aucune rotation, analyse humaine et éventuel abandon.
Changement de schéma	Élevée	Moyen	Fixtures, parseurs versionnés, compatibilité explicite et conservation du brut.
Données incomplètes	Élevée	Moyen	Score de complétude, champs obligatoires et comparaison avec d'autres sources.

Risque	Probabilité	Impact	Réponse
Erreur d'identité	Moyenne	Élevé	Mapping canonique, contrôles de compétition/participants/horaire et rejet des ambiguïtés.
Dépendance implicite du projet global	Faible si contrôlée	Élevé	Contrat d'export facultatif, tests de mode dégradé et règle ADR.
Fuite de secrets ou cookies	Faible	Élevé	Aucun cookie de navigateur, logs expurgés, secrets non versionnés.
Volume excessif	Faible	Moyen	Appels manuels, sémaphore 1, délai, cache et quotas internes.
Interprétation juridique erronée	Moyenne	Élevé	Avertissement explicite, relecture des conditions et absence de promotion automatique.

12.4 Journalisation

Les logs peuvent contenir : endpoint logique, clé de requête hachée, statut, latence, taille, hash du payload, version du parseur et erreur. Ils ne doivent pas contenir : payload complet, cookies, jetons, secrets, contenu de session ou en-têtes inutiles.

13. Stratégie de tests

13.1 Principe hors ligne d'abord

Les tests standards doivent être entièrement reproductibles sans Internet. Les snapshots enregistrés et minimisés deviennent des fixtures. L'objectif est de pouvoir corriger les parseurs, les mappings et les règles de cohérence sans rappeler la source.

13.2 Matrice de tests

Domaine	Scénarios essentiels
Parsing	Réponse nominale, champs absents, <code>null</code> , nouveaux champs, type numérique devenu texte, tableau vide, objet inattendu.
Transport	200, 304, 400, 401, 403, 429, 500, timeout, connexion refusée, contenu HTML, contenu trop grand.
Identité	Équipes inversées, compétition ambiguë, horaire corrigé, événement reporté, doublon d'identifiant.
Snapshots	Même hash, hash différent, ordre JSON différent mais contenu canonique identique, correction tardive.
Cohérence	Score négatif, tirs cadrés supérieurs aux tirs, possession incohérente, minute impossible, statut incompatible.
Persistence	Migration vide, reprise, transaction interrompue, contrainte d'unicité, purge et restauration.
Export	Schéma valide, champ obligatoire absent, écriture atomique, hash du manifeste, fichier déjà présent.
Interface	Validation de formulaire, confirmation d'appel, arrêt global, affichage des incidents et pagination.
Sécurité	Écoute locale, secret absent des logs, connecteur désactivé, refus d'un endpoint non autorisé.
Mode dégradé	Base accessible sans réseau, consultation des snapshots, export local et démarrage sans base URL.

13.3 Niveaux et outils

Niveau	Outil	But
Unitaire	JUnit / Mockito	Domaine, garde-fous, hash, mappers et validateurs.
Client HTTP	Starter RestClient Test, MockRestServiceServer ou serveur stub	Statuts, headers, timeouts et contenu inattendu.
Base	Testcontainers PostgreSQL	Migrations, JSONB, contraintes et transactions.
Web	MockMvc	Formulaires, routes, vues et erreurs.
Contrat	Fixtures versionnées	Compatibilité des parseurs et comparaison des schémas.
Réel	Profil Maven séparé	Qualification manuelle et bornée depuis Windows.

13.4 Séparation des tests réels

Aucun appel réel ne doit être déclenché par :

```
.\mvnw.cmd test
.\mvnw.cmd verify
```

Un test réel exige un profil explicite et une configuration locale :

```
.\mvnw.cmd verify -Psofascore-live-test
```

Le profil doit être inactif par défaut, échouer si l'activation n'est pas confirmée et limiter le nombre total de requêtes.

13.5 Critères de qualité des fixtures

- aucun secret, cookie ou identifiant de session ;
- payload réduit au strict nécessaire lorsque la licence ou la prudence l'impose ;
- date, endpoint logique, hash et statut documentés ;
- cas nominal et cas de rupture de schéma ;
- possibilité de reproduire le résultat du test sans la source externe.

14. Exploitation locale, supervision et sauvegarde

14.1 Health et métriques

Actuator fournit les points de santé et métriques de l'application. L'exposition reste locale et limitée aux endpoints nécessaires. Un indicateur personnalisé `sofascoreConnector` peut afficher :

- activation globale ;
- dernier succès et dernier échec ;
- circuit ouvert/fermé ;
- délai depuis le dernier appel ;
- nombre de 403/429 sur une fenêtre ;
- nombre d'incompatibilités de schéma ;
- disponibilité de la base et du dossier d'exports.

14.2 Procédures minimales

Procédure	Contenu
Démarrage	Vérifier Java 25, Docker/PostgreSQL, variables locales, migrations, port 8087 et connecteur désactivé.
Activation	Ouvrir l'interface, vérifier les garde-fous, activer explicitement et lancer un appel manuel de qualification.
Incident 403/429	Désactiver, capturer le statut et l'heure, ne pas changer d'adresse, analyser puis décider.
Schéma incompatible	Geler le parseur, conserver le brut, créer une fixture, corriger hors ligne et relancer les tests.
Purge	Exporter le manifeste, vérifier les fixtures utiles, purger selon la rétention et conserver les métadonnées.
Sauvegarde	pg_dump local chiffré si les données deviennent importantes, plus copie des exports et de la configuration non secrète.
Restauration	Recréer PostgreSQL, appliquer Flyway, restaurer, vérifier les hashes et exécuter la suite de tests.

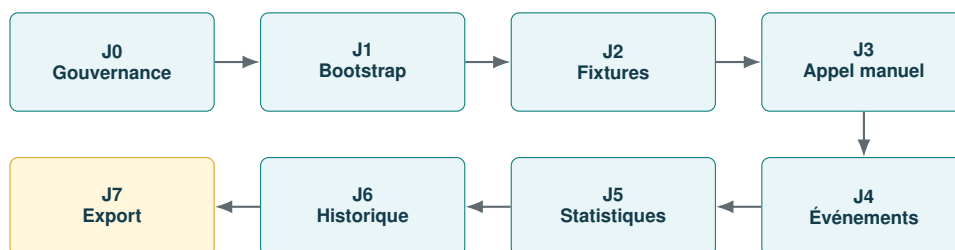
14.3 Mode dégradé

L'application doit démarrer et rester utilisable lorsque :

- la base URL n'est pas définie ;
- le connecteur est désactivé ;
- Internet est indisponible ;
- le circuit est ouvert ;
- un parseur est incompatible.

Dans ce mode, l'opérateur peut consulter les snapshots, comparer les données, corriger les mappings et produire des exports à partir des informations déjà validées.

15. Feuille de route



Jalon	Contenu	Preuve de sortie
J0	Gouvernance	ADR-SS-001, dépôt séparé, règles réseau, statut expérimental et non-objectifs.
J1	Bootstrap	Spring Boot, Java 25, UI locale, PostgreSQL, Flyway, Actuator et wrappers Maven.
J2	Fixtures	Parsing hors ligne d'une première famille de réponse et tests de schéma.
J3	Appel manuel	Une requête réelle, stockage brut, hash, affichage JSON, arrêt et incidents.
J4	Événements	Recherche par date, identité canonique locale et détail d'une rencontre.

Jalon	Contenu	Preuve de sortie
J5	Statistiques	Statistiques, incidents et compositions avec contrôles de complétude.
J6	Historique	Déduplication, snapshots, comparaison et corrections tardives.
J7	Export	Enveloppe JSON canonique, manifeste, provenance et validation humaine.
J8	Benchmark	Mesures de complétude, latence, stabilité, erreurs et coût en appels.
J9	Décision	Abandon, maintien local ou préparation d'une intégration facultative via nouvel ADR.

Condition avant le live

Le live n'est étudié qu'après J9. Il exige une décision spécifique sur les volumes, la fréquence, la fraîcheur, la pertinence, les conditions d'usage et les effets d'un blocage. Les résultats du prototype manuel ne suffisent pas à justifier un polling permanent.

16. Critères d'acceptation

ID	Critère
SS-AC-01	L'application compile, teste et s'exécute avec Java 25 LTS.
SS-AC-02	Elle démarre depuis Eclipse et avec <code>mvnw.cmd</code> .
SS-AC-03	Elle écoute uniquement sur <code>127.0.0.1</code> .
SS-AC-04	Le connecteur SofaScore est désactivé par défaut.
SS-AC-05	Une activation explicite est nécessaire avant tout appel externe.
SS-AC-06	La concurrence réseau maximale est égale à 1.
SS-AC-07	Le cache et le délai minimal sont vérifiés avant chaque appel.
SS-AC-08	Une requête manuelle conserve statut, horaires, latence, taille, hash et version de parseur.
SS-AC-09	Une réponse identique ne crée pas deux snapshots inutiles.
SS-AC-10	Un 403 suspend immédiatement le connecteur sans retry.
SS-AC-11	Un 429 respecte <code>Retry-After</code> lorsqu'il existe et interdit tout retry rapide.
SS-AC-12	Une réponse HTML inattendue ouvre le circuit et produit une alerte visible.
SS-AC-13	Un schéma inconnu est conservé brut et classé incompatible, sans normalisation silencieuse.
SS-AC-14	Aucun test standard Maven ne contacte Internet.
SS-AC-15	Les réponses nécessaires aux tests sont rejouables hors ligne.
SS-AC-16	Les données brutes et normalisées sont stockées séparément.
SS-AC-17	Chaque donnée normalisée référence son snapshot, son hash et son parseur.
SS-AC-18	Le mode d'acquisition est <code>DIRECT_LOCAL_ENDPOINT</code> , distinct d'une observation visuelle.
SS-AC-19	Chaque export porte une version de schéma, une provenance et un manifeste.
SS-AC-20	Les payloads bruts ne sont pas transmis automatiquement au Betting Project.
SS-AC-21	Aucun cookie, jeton ou secret de session n'est stocké ou journalisé.
SS-AC-22	Aucun proxy rotatif, navigateur automatisé ou mécanisme de contournement n'est présent.
SS-AC-23	L'arrêt du laboratoire n'a aucun effet sur la chaîne principale du Betting Project.
SS-AC-24	L'application reste exploitable hors ligne pour consulter, comparer et exporter les snapshots existants.
SS-AC-25	Les migrations PostgreSQL sont testées avec Testcontainers.
SS-AC-26	Une restauration de la base locale est documentée et vérifiée avant toute intégration future.
SS-AC-27	Le tableau de bord expose les derniers appels, incidents, circuits, erreurs de schéma et données à purger.

ID	Critère
SS-AC-28	Les conditions d'utilisation et la décision d'architecture sont revérifiées avant toute hausse de volume ou promotion.

16.1 Critères de décision après benchmark

Dimension	Mesure attendue
Accessibilité	Taux de succès, nature des refus, stabilité dans le temps et absence de challenge.
Complétude	Champs présents par endpoint, compétition, saison et état du match.
Exactitude	Identités, horaires, statuts, scores et corrections comparés à des sources de contrôle.
Fraîcheur	Heure source, heure de réception, retard médian/P95 et corrections tardives.
Stabilité	Erreurs HTTP, changements de schéma, compatibilité des parseurs et reprises.
Efficacité	Nombre d'appels nécessaires par dossier d'événement exploitable.
Maintenabilité	Temps nécessaire pour adapter un parseur et restaurer le fonctionnement hors ligne.
Risque	Fréquence des blocages, contraintes observées et dépendances techniques.
Valeur analytique	Données réellement uniques ou meilleures que les sources déjà retenues.

17. Décisions finales et livrables suivants

17.1 Décisions à figer

Sujet	Décision
Dépôt	betting-sofascore-local-lab, séparé du Betting Project.
Exécution	Windows local, application Java hors Docker au démarrage.
Java	Java 25 LTS pour compilation et runtime.
IDE	Eclipse 2026-06 (4.40.0) avec Spring Tools 5.3.0.
Framework	Spring Boot 4.1.0.
Client HTTP	RestClient.
Concurrence	Un appel SofaScore simultané au maximum.
Interface	Spring MVC + Thymeleaf sur 127.0.0.1:8087.
Base	PostgreSQL local dans Docker Desktop.
Migrations	Flyway.
Cache	Caffeine ; aucun Redis au MVP.
Réseau	Désactivé par défaut, appels manuels, arrêt sur refus/challenge.
Live	Hors MVP.
Données	Brut et normalisé séparés, hash, parseur et provenance obligatoires.
Intégration	Export JSON puis push sortant éventuel après nouvel ADR.
Statut	Expérimental, local, non approuvé production, non critique.

17.2 Livrables suivants

1. bootstrap du dépôt et du projet Spring Initializr équivalent ;
2. README.md, AGENTS.md et conventions de sécurité ;

3. Docker Compose PostgreSQL local et première migration Flyway ;
4. propriétés typées `SofascoreProperties` avec validation ;
5. tableau de bord local avec connecteur désactivé ;
6. catalogue logique vide ou limité à un premier endpoint de qualification ;
7. fixtures hors ligne et tests de parseur ;
8. Work Order J1/J2 avec critères d'acceptation ;
9. rapport de benchmark après les premières mesures.

Conclusion

Le laboratoire est autorisé comme complément expérimental du Betting Project, à condition de rester local, isolé, manuel et réversible. Java 25 LTS devient la cible unique. Les résultats doivent être évalués comme des preuves de benchmark et non comme une validation automatique de production ou de conformité contractuelle.

18. Annexe A - Exemple de modèle de données

Table	Champs principaux indicatifs
provider_snapshot	id, provider, logical_endpoint, request_key, requested_at, received_at, http_status, content_type, latency_ms, payload_jsonb, payload_sha256, parser_version, schema_status, error_code.
competition	id, provider_competition_id, canonical_name, country, season, active.
participant	id, provider_participant_id, canonical_name, type, country.
event	id, provider_event_id, competition_id, start_at, status, home_participant_id, away_participant_id, score, version.
event_metric	id, event_id, metric_code, metric_value, metric_unit, period, observed_at, snapshot_id, confidence.
event_incident	id, event_id, incident_type, minute, participant_id, player_id, payload, snapshot_id.
event_lineup	id, event_id, participant_id, player_id, starter, position, shirt_number, snapshot_id.
provider_entity_mapping	provider, entity_type, provider_id, local_id, confidence, valid_from, valid_to.
export_manifest	id, schema_version, export_path, content_sha256, created_at, validation_status, source_snapshot_ids.

Règle de provenance

Une métrique sans `snapshot_id`, sans heure de réception et sans version de parseur n'est pas exploitable par le laboratoire. La normalisation doit renforcer la traçabilité, pas effacer l'origine.

19. Annexe B - Exemples de configurations et commandes

19.1 Variables d'environnement locales

```
SOFAScore_DB_URL=jdbc:postgresql://localhost:5432/sofascore_local_lab
SOFAScore_DB_USER=sofascore_lab
SOFAScore_DB_PASSWORD=***
SOFAScore_BASE_URL=***
SOFAScore_EXPORT_DIR=C:\BettingProject\SofaScore\exports
```

La base URL n'est pas versionnée. Le projet peut démarrer sans elle et doit rester en mode hors ligne.

19.2 Commandes de contrôle

```
# Version Java
java -version

# Build complet hors ligne reseau fournisseur
.\mvnw.cmd clean verify

# Demarrage local
.\mvnw.cmd spring-boot:run -Dspring-boot.run.profiles=local

# Test reel, uniquement apres activation explicite
.\mvnw.cmd verify -Psofascore-live-test
```

19.3 Arborescence documentaire

```
betting-sofascore-local-lab/
|-- ADR-SS-001-experimentation-endpoints-sofascore-depuis-windows.md
|-- AGENTS.md
|-- README.md
|-- docs/
|   |-- architecture/
|   |-- benchmark/
|   |-- runbooks/
|   '-- work_orders/
|-- fixtures/
|   |-- scheduled-events/
|   |-- event-details/
|   '-- schema-breaks/
|-- infra/
|   '-- compose.yaml
|-- exports/
|-- src/
|-- pom.xml
|-- mvnw
'-- mvnw.cmd
```

20. Annexe C - Références

- R0** *Projet de solution d'aide à la décision pour les paris sportifs - Document de cadrage v0.5.1*, 7 août 2026. Sections particulièrement utilisées : 2.3, 3.1 à 3.5, 4.3, 8.8, 11, 15.1 et annexe B.
- R1** Oracle, *Java SE Support Roadmap*. Java 25 figure parmi les versions LTS.
<https://www.oracle.com/java/technologies/java-se-support-roadmap.html>
- R2** Oracle, *JDK Releases / Java Release Support Timeline*. Java 26 appartient au cycle non-LTS ; Java 25 LTS dispose d'un horizon de support long.
<https://www.oracle.com/java/technologies/update-release-numbers-change.html>
- R3** Spring, *Spring Boot 4.1.0 - System Requirements*. Java 17 minimum, compatibilité jusqu'à Java 26 incluse ; Maven 3.6.3 ou supérieur.
<https://docs.spring.io/spring-boot/system-requirements.html>
- R4** Spring Tools, *Previous Versions*. Distribution Spring Tools 5.3.0 pour Eclipse 2026-06 (4.40).
<https://github.com/spring-projects/spring-tools/wiki/Previous-Versions>
- R5** Eclipse Foundation, *Eclipse 4.40 - New and Noteworthy* et packages Eclipse IDE 2026-06.
<https://www.eclipse.org/eclipse/news/4.40/>
<https://www.eclipse.org/downloads/packages/>
- R6** Spring Framework, *REST Clients*. RestClient est le client synchrone recommandé ; RestTemplate est déprécié à partir de Spring Framework 7.
<https://docs.spring.io/spring-framework/reference/integration/rest-clients.html>
- R7** Spring Boot, *Task Execution and Scheduling*. Activation des threads virtuels avec la propriété `spring.threads.virtual.enabled=true` sur Java 21+.
<https://docs.spring.io/spring-boot/reference/features/task-execution-and-scheduling.html>
- R8** Spring Boot, *Build Systems*. Liste des starters 4.1, notamment webmvc, restclient, flyway, data-jpa et actuator.
<https://docs.spring.io/spring-boot/reference/using/build-systems.html>
- R9** SofaScore, *Terms & Conditions*. Référence à relire avant chaque évolution du périmètre.
<https://www.sofascore.com/terms-and-conditions>
- R10** SofaScore, *Sports Stats and Data on Your Website - Widgets*. Les widgets sont des composants d'affichage intégrables.
<https://corporate.sofascore.com/widgets>
- R11** JUnit, documentation du framework de test. La version exacte doit rester gérée par le BOM Spring Boot sauf besoin démontré.
<https://junit.org/>

Volatilité des références

Les versions, conditions, structures d'endpoint et pages officielles peuvent évoluer. Les références ont été contrôlées le 8 août 2026. Le document doit être révisé avant tout jalon modifiant le volume, le mode d'accès ou l'hébergement.